

Acceso a Datos

Manejo de Ficheros

Tema 2

1. Introducción

Datos persistentes Datos que se guardan en ficheros y persisten **más allá de la ejecución** de la aplicación que los trata, almacenados en unidades de almacenamiento secundario.

Java ofrece las clases necesarias para gestionar las operaciones de entrada y salida en la librería `java.io`. Esta librería contiene clases para **leer y escribir entradas** en un flujo de datos, **operar con ficheros** en el sistema de ficheros local, y gestionar la **serialización** de objetos.

Desde la versión 14 también se ofrece la librería `java.nio`, una versión non-blocking, que permite manejar **múltiples canales con uno o varios hilos**. El procesamiento de datos es más complicado, pero facilita gestionar múltiples conexiones que manejan una pequeña cantidad de datos.

2. Clases asociadas a las operaciones de gestión de ficheros y directorios

2.1. Clase File

```
1 File f = new File("ejemplo.txt");
```

Proporciona una representación abstracta de ficheros y directorios. Las instancias que crea representan **nombres de archivos**, no archivos en si mismos.

Un objeto `File` permite examinar el nombre del archivo, descomponerlo en la rama de directorios, o crear el archivo si no existe mediante `FileWriter(File f)`.

También permite realizar las siguientes operaciones sobre el archivo:

Comprobar si existe mediante el método estático `exists()`.

Renombrarlo mediante `renameTo()`.

Borrarlo mediante `delete()` o `deleteOnExit()`.

Crear un nuevo fichero temporal con un nombre único mediante `createTempFile()`.

Establecer la fecha y hora de modificación con `setLastModified()`.

Crear un directorio mediante `mkdir()` o `makedirs()`.

Listar el contenido de un directorio mediante `list()` y `listfiles()`.

Listar los nombres de archivo de la raíz del sistema mediante el método estático `listRoots()`.

2.2. Interfaz FilenameFilter

Puede usarse para crear filtros que establezcan **criterios de filtrado** relativos al nombre de los ficheros.

La clase que lo implemente incluirá el método boolean `accept(File dir, String name)`, que devolverá `true` en caso de que el fichero cuyo nombre se indica en el parámetro `name` **aparezca en la lista de ficheros** del directorio `dir`.

El siguiente ejemplo de código implementa la interfaz para crear una clase que filtra los archivos de un directorio que acaben en una extensión específica.

```
1 import java.io.File;
2 import java.io FilenameFilter;
3
4 public class Filtrar implements FilenameFilter {
5     String extension;
6     Filtrar (String extension) {
7         this.extension = extension;
8     }
9     public boolean accept(File dir, String name) {
10         return name.endsWith(extension);
11     }
12 }
13
14 public static void main (String[] args)
15 {
16     File f = new File("C:\\datos")
17     String[] lista = f.list();
18     lista = f.list(new Filtrar(".txt"));
19     for(int c = 0; c < lista.length; c++) {
20         System.out.println(lista[c]);
21     }
22 }
```

3. Flujos

Flujo / Stream Abstracción de **todo aquello que produce o consume información**. El sistema de E/S de Java vincula el flujo al dispositivo físico.

Java define dos tipos de flujos:

Byte streams (8 bits) Orientados a la lectura y escritura de **datos binarios**.
Determinado por dos clases abstractas, `InputStream` y `OutputStream`.

Character streams (16 bits) Determinados por las clases abstractas `Reader` y `Writer`, que manejan flujos de **caracteres Unicode**.

3.1. Flujos basados en bytes

Las clases que heredan de `OutputStream`, para la escritura de **ficheros binarios**, son:

FileOutputStream Escribe **bytes** en un fichero.

ObjectOutputStream Convierte **objetos y variables** en vectores de bytes.

DataOutputStream Da formato a los **tipos primitivos y objetos String**.
Todos los métodos empiezan por `write`: `writeByte`, `writeFloat`, etc.

Las clases que heredan de `InputStream`, para la lectura de ficheros binarios, son:

FileInputStream Lee bytes de un fichero.

ObjectInputStream Convierte en **objetos y variables** los vectores de bytes
leídos de un `InputStream`.

3.2. Flujos basados en caracteres

`FileInputStream`, `FileOutputStream`, `FileReader` y `FileWriter` realizan la escritura/lectura **directamente en el disco duro** cada vez que hay un cambio, lo que hace que cambios frecuentes sean lentos debido al continuo acceso al disco duro.

`BufferedReader`, `BufferedInputStream`, `BufferedWriter` y `BufferedOutputStream` añaden un flujo intermedio que controla los accesos a disco: la información **se almacena en el *buffer*** y no se guardarán los datos en el disco duro hasta que no sean **manualmente** guardados.

Al leer, la clase **leerá más datos de los que se hayan pedido**, y devolverá los datos almacenados en las siguientes lecturas, hasta que necesite volver a leer del disco duro. Esto hace el acceso al disco más **rápido y eficiente**.

El siguiente ejemplo lee caracteres de un archivo hasta llegar al final de este, delimitado por el carácter EOF.

```
1 public static void main (String args[]) throws IOException {  
2     BufferedReader br = new BufferedReader(new FileReader("datos.txt"));  
3     int cod = br.read();  
4     char character;  
5     while (codigo != -1) {  
6         character = (char) codigo;  
7         System.out.println(character);  
8         codigo = br.read();  
9     }  
10 }
```

4. Formas de acceso a un fichero

Acceso aleatorio Permite acceder a los datos en **forma no secuencial**. Obliga a que el archivo esté **disponible en su totalidad** en el momento de ser accedido. Java proporciona la clase `RandomAccessFile` para este tipo de acceso.

Acceso secuencial Los datos se leen desde el comienzo del archivo hasta el final, que **no necesita conocerse a priori**. La lectura del teclado o la escritura en una consola son ejemplos de acceso secuencial.

Concatenación (tuberías o pipes) Conexiones entre programas que se ejecutan simultáneamente dentro de una misma máquina: **lo que uno produce es recibido por otro**.

5. Java NIO

Librería que define interfaces y clases para que la máquina virtual Java tenga acceso a **archivos, atributos de archivos y sistemas de archivos** sin bloqueo.

A diferencia de la API estándar, que trabaja con secuencias de bytes y caracteres, **NIO trabaja con canales y buffers**: los datos siempre se leen de un canal a un búfer y viceversa.

Selector Objeto que puede gestionar **múltiples canales para eventos**. Permite que un solo hilo maneje múltiples canales.

5.1. NIO Path

java.nio.file.Path Interfaz que representa un *path* y puede utilizarse para **localizar ficheros** en el sistema de archivos.

java.nio.file.FileSystem Define un **sistema de ficheros completo** en lugar de un solo directorio o fichero.

La forma más sencilla de construir un objeto que cumple esta interfaz es a partir de la clase `java.nio.file.Paths`, que tiene objetos estáticos que devuelven objetos `Path` a partir de una representación `String` de la ruta deseada.

```
1 Path p = Paths.get("C:\\fichero.txt")
```

`Path` permite:

- **Recuperar** partes de una ruta.
- **Eliminar** redundancias de una ruta.
- **Convertir** una ruta.
- **Unir** dos rutas.
- **Crear** una ruta relativa a otra dada.
- **Comparar** dos rutas. Una instancia de tipo `Path` refleja el sistema de nombrado del sistema operativo, por lo que objetos `Path` de diferentes sistemas operativos **no pueden ser comparados** entre si.

5.2. NIO Files

Clase en la ruta `java.nio.file.Files` que actúa como el otro punto de entrada a la librería de ficheros. Permite:

Verificar permisos mediante `exists()`, `isReadable()`, `isWritable()` y `isExecutable()`.

Crear y borrar directorios mediante `delete()`, `createFile()` y `createDirectory()`.

Copiar ficheros mediante `copy()`.

Mover ficheros y directorios mediante `move()`.

5.3. Escribir contenido en un fichero

5.3.1. Desde arrays de bytes

La forma más sencilla, y limitada, de escritura de ficheros, mediante `java.nio.file.Files.write()`

```
1 import java.nio.IOException;
2 import java.nio.file.Files;
3 import java.nio.file.Path;
4 import java.nio.file.Paths;
5 import java.nio.file.StandardOpenOption;
6
7 public class Ejemplo {
8     public static void main(String args[]) {
9         Path inputFile = Paths.get("origen.txt");
10        Path outputFile = Paths.get("destino.txt");
11        try {
12            byte[] contents = Files.readAllBytes(inputFile);
13            Files.write(outputFile, contents, StandardOpenOption.WRITE,
14                StandardOpenOption.CREATE, StandardOpenOption.TRUNCATE_EXISTING);
15        } catch (IOException e) {
16            System.err.println("ERROR: " + e);
17            System.exit(1);
18        }
19    }
20 }
```

5.3.2. Desde buffers

```
1 import java.nio.file.Path;
2 import java.nio.file.Paths;
3 import java.nio.file.Files;
4 import java.io.IOException;
5 import java.nio.charset.Charset;
6 import java.io.BufferedReader;
7 import java.io.BufferedWriter;
8 import java.nio.file.StandardOpenOption;
9
10 public class Ejemplo {
11     public static void main(String [] args) {
12         path input = Paths.get("origen.txt");
13         path output = Paths.get("destino.txt");
14         try {
15             BufferedReader inputReader = Files.newBufferedReader(input,
16                 Charset.defaultCharset());
17             BufferedWriter outputWriter = Files.newBufferedWriter(output,
18                 Charset.defaultCharset(),
19                 StandardOpenOption.WRITE,
20                 StandardOpenOption.CREATE,
21                 StandardOpenOption.TRUNCATE_EXISTING);
22             String line;
23             while ((line = inputReader.readLine()) != null) {
24                 outputWriter.write(line, 0, line.length());
25                 outputWriter.newLine();
26             }
27             inputReader.close();
28             outputWriter.close();
29         } catch (IOException e) {
30             System.err.println("Error" + e);
31         }
32     }
33 }
```

```
31     System.exit(1);  
32   }  
33 }  
34 }
```

6. Trabajo con ficheros XML

6.1. Conceptos previos

Parsing Análisis léxico-sintáctico. Proceso mediante el cual **se lee y analiza un documento XML para comprobar que está bien formado** y pasarlo a una aplicación cliente que necesite consumirlo.

Schema Especificación XML que **dicta los componentes permitidos de un documento XML y las relaciones entre los componentes**. Un documento XML no tiene por qué tener un esquema, pero si lo tiene, debe atenerse a ese esquema para ser **válido**.

6.2. Definiciones

JAXB Java Architecture for XML Binding. Simplifica el acceso a documentos XML **representando la información obtenida de los documentos XML en un programa en formato Java**. Proporciona métodos para, a partir de documentos XML, obtener árboles de contenido para operar con ellos o manipularlos en Java. JAXB se menciona con más detalle en la sección [sección 6.6](#).

Parsear Analizar el documento y **dividirlo o separarlo lógicamente** en piezas discretas.

Binding Generar un conjunto de **clases Java que representan un esquema XML**.

Compilador de esquema Liga un esquema fuente a un conjunto de elementos de programa.

Binding routine framework Proporciona operaciones de **marshalling** y **un-marshalling** para **acceder, manipular y validar contenido XML** usando un esquema o elementos de programa.

Marshalling Proceso de **codificación de un objeto en un medio de almacenamiento**, normalmente un fichero. Proporciona la capacidad para convertir un árbol de objetos JAXB a ficheros XML.

Unmarshalling Proporciona a aplicaciones cliente la capacidad de convertir **datos XML a objetos JAXB**.

6.3. DOM

Document Object Model. Define una interfaz que permite a los programas **acceder y actualizar el estilo, la estructura y el contenido** de los documentos XML.

El estándar W3C define la clase `DocumentBuilder` para instanciar estructuras DOM a partir de un XML.

```
1 DocumentBuilderFactory dbFactory = DocumentBuilderFactory.newInstance();
2 DocumentBuilder dBuilder = dbFactory.newDocumentBuilder();
3 Document doc = dBuilder.parse (new File("fitxer.xml"));
```

La escritura de la información contenida en el DOM se puede secuenciar en forma de texto mediante **Transformer**, utilidad que permite realizar **conversiones entre representaciones de información jerárquica**.

```
1 Transformer trans = TransformerFactory.newInstance().newTransformer();
2 StreamResult result = new StreamResult(file);
3 DOMSource source = new DOMSource(doc);
4 trans.transform (source,result);
```

Además de `StreamResult` y `DOMSource`, podemos usar `DOMResult`, `SAXResult` o `StreamResult` y `DOMSource`, `SAXSource` o `StreamSource`, según el contenedor destino y el origen.

6.3.1. La estructura DOM

Toma la forma de un árbol donde cada parte del XML es representada en forma de nodo.

Documento Nodo principal que representa **todo el documento XML**.

Nodos elemento Las etiquetas del documento, incluida la **etiqueta raíz**.

Nodo TextElement Contenido textual de una etiqueta.

Nodo Attribute Nodos que representan atributos.

El DOM resultante de un XML termina siendo una copia exacta del archivo, pero con una disposición distinta.

Los retornos de carro se interpretan en DOM como un **hijo**.

El **contenido textual** de las etiquetas se plasma en el DOM como un nodo hijo de la etiqueta contenedora: para obtener el texto de una etiqueta, hay que obtener el **primer hijo** de esta.

La interfaz `Document` ofrece un conjunto de métodos para seleccionar diferentes partes del árbol **a partir del nombre de la etiqueta o de un atributo**, y devuelve objetos `Element`, representando un nodo y todos sus hijos.

Los objetos `Element` permiten añadir **nuevos hijos** (`appendChild`), **asignar valores** a atributos (`setAttribute`), **consultar** atributos (`getAttribute`) o navegar por los nodos del árbol.

Podemos crear nuevos elementos mediante `createElement`, `createTextNode` o `createComment`. Crear un nodo requiere añadirlo posteriormente al árbol, **asignándole un padre** mediante `appendChild`.

Como ejemplo, considérese el siguiente documento XML.

```
1 <?xml version = "1.0"?>
2 <clase>
3   <alumno numero = "393">
4     <nombre> Luis </nombre>
5     <apellido> Luna </apellido>
6     <apodo> Na </apodo>
7     <marcas> 85 </marcas>
8   </ alumno>
9   <alumno numero = "493">
10    <nombre> Antonio </nombre>
11    <apellido> Alvarez</apellido>
12    <apodo> Avez </apodo>
13    <marcas> 95 </marcas>
14  </ alumno>
15  <alumno numero = "593">
16    <nombre> Juan </nombre>
17    <apellido> Juz </apellido>
18    <apodo> jazz </apodo>
19    <marcas> 90 </marcas>
20  </alumno>
21 </clase>
```

Puede ser leído y convertido al DOM mediante el siguiente programa.

```
1 import java.io.File;
2 import javax.xml.parsers.DocumentBuilderFactory;
3 import javax.xml.parsers.DocumentBuilder;
4 import org.w3c.dom.Document;
5 import org.w3c.dom.NodeList;
6 import org.w3c.dom.Node;
7 import org.w3c.dom.Element;
8
9 public class DomParserDemo {
10   public static void main(String[] args) {
11     try {
12       File inputFile = new File("clase.xml");
13       DocumentBuilderFactory dbFactory = DocumentBuilderFactory.newInstance();
14       DocumentBuilder dBuilder = dbFactory.newDocumentBuilder();
```

```

15     Document doc = dBuilder.parse(inputFile);
16     doc.getDocumentElement().normalize();
17     System.out.println("Root element : " +
18     doc.getDocumentElement().getNodeName());
19     NodeList nList = doc.getElementsByTagName("alumno");
20     System.out.println("-----");
21     for (int temp = 0; temp < nList.getLength(); temp++) {
22         Node nNode = nList.item(temp);
23         System.out.println("\nCurrent Element : " + nNode.getNodeName());
24         if (nNode.getNodeType() == Node.ELEMENT_NODE) {
25             Element eElement = (Element) nNode;
26             System.out.println("numero de alumno : " + eElement.getAttribute("numero"));
27             System.out.println("nombre : " + eElement.getElementsByTagName("nombre").
28             item(0).getTextContent());
29             System.out.println("apellido : " + eElement.getElementsByTagName("apellido").
30             item(0).getTextContent());
31             System.out.println("apodo : " + eElement.getElementsByTagName("apodo").item(0).
32             getTextContent());
33             System.out.println("marcas : " + eElement.getElementsByTagName("marcas").item(0).
34             getTextContent());
35         }
36     }
37 } catch (Exception e) {
38     e.printStackTrace();
39 }
40 }
41 }

```

6.4. SAX

Interfaz simple de aplicaciones XML, fácil e intuitiva. **Lee un fichero XML de forma secuencial, produciendo eventos en función de los resultados que encuentra.** Cada evento invoca a un método definido por el programador.

Procesa el documento poco a poco y **apenas consume memoria**, pero impide tener una visión general de todo el documento, a diferencia de DOM, que carga todo el documento de golpe en memoria.

Es conveniente usar SAX cuando el documento es considerablemente grande, no requiere una modificación estructural, o se busca parsear diferentes partes del mismo.

Solo puede leer archivos XML, no escribirlos.

SAX genera un procesador de XML llamado XMLReader que implementa las siguientes interfaces.

ContentHandler Recibe las notificaciones de eventos.

DTDHandler Recoge los eventos de DTD.

ErrorHandler Crea el tratamiento de errores.

EntityResolver Usado para referenciar a otra entidad.

DefaultHandler Implementa los métodos que el programador definirá:

startDocument Cuando el documento empieza.

endDocument Cuando el documento ha acabado.

startElement Cuando encuentra un nuevo elemento, nodo, etiqueta, tag, etc.

endElement Llamado una vez se ha leído el documento.

Characters Se invoca para encontrar cadenas de texto.

6.5. StAX

API basada en Java para analizar documentos XML. A diferencia de SAX, en el que **el analizador notifica a la aplicación cliente** cuando la información está disponible, en StAX es **la aplicación cliente quien debe solicitar la información al analizador**.

Puede leer y escribir XML.

Consta de dos APIs:

Cursor Representa un puntero con el que se puede ir **hacia delante** en un documento desde el principio hacia el final. Puede apuntar a un solo elemento a la vez.

Iterator Representa el flujo del documento como un **conjunto de eventos** discretos emitidos en el mismo orden el que el parser los proporciona al leerlos del documento. Por cada paso, ofrece un objeto **XMLEvent** que contiene información sobre el evento generado.

Es conveniente usarlo cuando el documento no está profundamente anidado, es muy grande y consumiría demasiada memoria en DOM, o el problema solo involucra una parte del documento.

Los eventos de atributo se tratan como eventos secundarios. Los de espacio de nombres aparecen **dos veces** en la secuencia de eventos.

```
1 XMLInputFactory xmlif = XMLInputFactory.newInstance();
2 XMLStreamReader xmlsr =xmlif.createXMLStreamReader(new FileReader("books.xml"));
3
4 ----o bien----
5
6 XMLInputFactory inputFactory = XMLInputFactory.newInstance();
7 InputStream in = new FileInputStream("books.xml");
8
```

6.6. Binding

Técnica que consiste en **vincular clases Java con formatos específicos de almacenamiento** de manera automatizada.

Como se mencionó en la sección [sección 6.2](#), Java Architecture for XML Binding (JAXB) permite a los desarrolladores Java asignar clases Java a representaciones XML, **serializando Java a XML y a la inversa**.

6.6.1. Configuración con anotaciones

Anotaciones Clases que pueden asociarse a un **paquete, clase, atributo o parámetro**, declaradas en el código anteponiendo el símbolo @ al nombre de la anotación. Cuando el compilador detecta una anotación, crea una instancia de la clase y la inyecta en el elemento afectado.

Pueden declararse **con parámetros** (que pueden ser otras anotaciones, valores constantes o valores literales) o **sin ellos**.

@XMLRootElement Define la raíz del XML.

@XMLType Permite definir en qué orden se van a escribir los elementos dentro del XML. Deben tenerla todas las clases que no mapean la raíz.

@XMLElement Define el elemento XML que se va a utilizar. Debe usarse antes del método **setter**.

@XMLElementWrapper Ofrece la posibilidad de crear un contenedor alrededor de una representación XML.

6.7. Funcionamiento de JAXB

1. **Escribir el esquema**, documento XML que contiene la estructura que se tomará como indicaciones para construir las clases.
2. **Generar los ficheros fuente de Java**. El compilador tomará el esquema como entrada de información y compilará el código fuente.
3. **Construir el árbol de clases Java** instanciando las clases generadas o invocando el método **unmarshall** de estas.
4. **Acceder al árbol de contenido** usando la aplicación.
5. **Generar un documento XML** desde el árbol de contenido invocando el método **marshall** sobre el objeto raíz.

6.7.1. Generación automática de clases Java a partir del esquema xsd

xjc Herramienta que se puede utilizar para **convertir un XML Schema a representaciones de clase Java**.

schemagen Realiza el proceso inverso, **creando un Schema a partir de un conjunto de clases anotadas**.

7. Librerías para conversión de documentos XML a otros formatos

7.1. Introducción a JasperReports

JasperReports Herramienta que consta de un motor para la generación de informes. Empaquetada en formato JAR y puede ser utilizada como librería.

Las plantillas de los informes son **ficheros XML** con la extensión **.jrxml**, que siguen estas secciones.

title Título del informe.

pageHeader Encabezado del documento.

columnHeader Encabezado de las columnas.

detail Cuerpo del documento.

columnFooter Pie de la columna.

pageFooter Pie del documento.

summary Cierre del documento.

Los pasos a seguir son

1. **Diseñar la plantilla del informe**, un fichero **.jrxml** que sigue la estructura de un archivo DTD validado mediante `<!DOCTYPE jasperReport PUBLIC //JasperReports//DTD Report Design//ENhttp://jasperreports.sourceforge.net/dtds/jasperreport.dtd"`

2. **Compilación.** Con el diseño realizado, se compila mediante el método `compileReport`. Esto transforma el diseño en un objeto serializable `net.sf.jasperreports.engine.JasperReport`.
3. **Rellenar el informe con datos** mediante los métodos `fillReport`, pasando como parámetros el objeto de diseño y la conexión a la base de datos. Esto devuelve un documento listo para ser impreso, de tipo `JasperReport`.
4. **Visualización.** Podemos mostrarlo en pantalla mediante `JasperViewer`, imprimirlo mediante `JasperPrintManager` o exportarlo mediante los métodos `exportReport`.

© 2026 - Esteban N. - estnicolas@proton.me

Este texto se encuentra licenciado bajo Creative Commons
Atribución-Reconocimiento/NoComercial-CompartirIgual 4.0 Internacional.

Para ver una copia de esta licencia visite:

<https://creativecommons.org/licenses/by-nc-sa/4.0/legalcode.es>.



Ninguna parte de este documento, código o texto, ha sido realizada, apoyada, consultada, generada, corregida o involucrada en ninguna forma o contexto con inteligencia artificial, ni lo será en cualquier futura posible versión o corrección.

<https://notbyai.fyi>